

To install a Python library, you can use `pip install` followed by the library name. For example, to install `requests`:

```
# =====  
# HOUGH TRANSFORM IN NOISY IMAGES  
# =====  
  
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from google.colab import files  
  
# -----  
# 1. Upload Image  
# -----  
uploaded = files.upload()  
filename = list(uploaded.keys())[0]  
  
img = cv2.imread(filename)  
  
# Convert to grayscale  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
  
# -----  
# 2. Add Artificial Noise  
# -----  
noise = np.random.normal(0, 35, gray.shape)  
noisy = gray.astype(np.float32) + noise  
noisy = np.clip(noisy, 0, 255).astype(np.uint8)  
  
# -----  
# 3. Canny Edge Detection on Noisy Image  
# -----  
edges_noisy = cv2.Canny(noisy, 50, 150)  
  
# -----
```

```
# 4. Hough Transform on Noisy Image
# -----
lines_noisy = cv2.HoughLinesP(
    edges_noisy,
    rho=1,
    theta=np.pi / 180,
    threshold=30,
    minLineLength=20,
    maxLineGap=10
)

result_noisy = cv2.cvtColor(noisy, cv2.COLOR_GRAY2BGR)

if lines_noisy is not None:
    for line in lines_noisy:
        x1, y1, x2, y2 = line

        cv2.line(
            result_noisy,
            (x1, y1),
            (x2, y2),
            (0, 0, 255),
            2
        )

# -----
# 5. PREPROCESSING
# -----

# Gaussian filtering to reduce noise
filtered = cv2.GaussianBlur(
    noisy,
    (5, 5),
    0
)

# -----
# 6. Canny After Preprocessing
```

```
# -----
edges_clean = cv2.Canny(
    filtered,
    50,
    150
)

# -----
# 7. Morphological Closing
# -----
kernel = np.ones((3, 3), np.uint8)

edges_morph = cv2.morphologyEx(
    edges_clean,
    cv2.MORPH_CLOSE,
    kernel
)

# -----
# 8. Improved Hough Transform
# -----
lines_clean = cv2.HoughLinesP(
    edges_morph,
    rho=1,
    theta=np.pi / 180,
    threshold=80,      # Higher threshold
    minLineLength=50,  # Ignore short false lines
    maxLineGap=15      # Connect small gaps
)

result_clean = cv2.cvtColor(
    filtered,
    cv2.COLOR_GRAY2BGR
)

# Draw detected lines
if lines_clean is not None:
    for line in lines_clean:
```

```
for line in lines_clean:
    x1, y1, x2, y2 = line

    cv2.line(
        result_clean,
        (x1, y1),
        (x2, y2),
        (0, 255, 0),
        3
    )

# -----
# 9. Display Results
# -----

plt.figure(figsize=(16, 10))

plt.subplot(2, 4, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(noisy, cmap="gray")
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(edges_noisy, cmap="gray")
plt.title("Edges from Noisy Image")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(cv2.cvtColor(result_noisy, cv2.COLOR_BGR2RGB))
plt.title("Hough on Noisy Image")
plt.axis("off")

plt.subplot(2, 4, 5)
```

```
plt.imshow(filtered, cmap= gray )
plt.title("After Gaussian Filtering")
plt.axis("off")

plt.subplot(2, 4, 6)
plt.imshow(edges_clean, cmap="gray")
plt.title("Clean Canny Edges")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(edges_morph, cmap="gray")
plt.title("Morphological Closing")
plt.axis("off")

plt.subplot(2, 4, 8)
plt.imshow(cv2.cvtColor(result_clean, cv2.COLOR_BGR2RGB))
plt.title("Improved Hough Detection")
plt.axis("off")

plt.tight_layout()
plt.show()

# -----
# 10. Display Number of Detected Lines
# -----

noisy_count = 0 if lines_noisy is None else len(lines_noisy)
clean_count = 0 if lines_clean is None else len(lines_clean)

print("Lines detected before preprocessing :", noisy_count)
print("Lines detected after preprocessing :", clean_count)
```

Choose Files Screenshot... 084833.png

Screenshot 2026-05-09 084833.png(image/png) - 676711 bytes, last modified: 5/9/2026 - 100% done

Saving Screenshot 2026-05-09 084833.png to Screenshot 2026-05-09 084833 (1).png



